Exp: 1 Create a Kanban Board to Visualize the Tasks.

• Create Columns for To Do, In-Progress and Done.

• Add at least 5 Sample Tasks

• Move the Tasks across the Columns to Simulate the Workflow

Exp 2: Sketch a Simple Prototype of a Bus Ticket Booking System using Figma Tool

Exp 3: The stakeholders have conflicting views on the user interface design for an E-Commerce mobile app. Create a prototype using Figma tool to discuss with the stakeholders to get their feedback and approval.

Exp 4: The string is a sequence of characters placed in double quotes (" "). Performing different operations on string data is called String Handling. Strings are immutable. Whenever a change to a String is made, an entirely new String is created. If we want to store a group of characters we can use a char array. String provides various methods to perform different operations on strings. Using Raptor draw a flowchart to display the total number of characters in the string and returns it.

Exp 5: Using Raptor – Draw and validate a flowchart for a given string of lower case English alphabets. One can choose any two characters in the string and replace all the occurrences of the first character with the second character and replace all the occurrences of the second character with the first character. Using Raptor draw and validate the flowchart lexicographically smallest string that can be obtained by doing this operation at most once.

Exp 6: An online food delivery platform enables customers to browse restaurants, view menus, place food orders, make online payments, and track delivery status. Restaurants use the system to manage their menus and update order status, while delivery agents are responsible for accepting delivery requests and updating delivery progress. Design a Class Diagram, Use case Diagram and Activity Diagram for the Online Food Delivery System showing the relevant classes, their attributes, methods, and relationships.

Exp 7: A college employs a large number of security personnel who are assigned duties at various locations across the campus. Each security person follows a duty schedule that includes date, duty start time, duty end time, and assigned location, ensuring that every campus location is covered. If a security person takes leave, it is recorded manually, and at the end of each month, salaries are calculated based on duties performed and leaves taken.

Since manual handling of duty allocation, leave management, and salary calculation is time-consuming and complex, the college plans to develop an Online Security Management System to automate these activities. Develop Class Diagram, Use case Diagram and Activity Diagram to represent the workflow of the Online Security Management System.

Exp 8: Cyclomatic Complexity in Software Testing is a testing metric used for measuring the complexity of a software program. It is a quantitative measure of independent paths in the source code of a software program. Cyclomatic complexity can be calculated by using control flow graphs or with respect to functions, modules, methods or classes within a software program. Cyclomatic complexity for a flow graph G is V(G)=E-N+2. Cyclomatic complexity is a software metric (measurement), used to indicate the complexity of a program, Cyclomatic complexity = E - N + P where, E = number of edges in the flow graph, N = number of nodes in the flow graph, P = number of nodes that have exit points. Find Cyclomatic Complexity for a graph having number of edges as 17, number of nodes as 13 and number of predicate nodes in the flow graph as 5.

EXP 9: Create a Scrum Project in Jira.

• Add a backlog with at least 5 items (e.g., "Create user registration page", "Develop API for login").

• Prioritize the backlog and create a 1-week sprint.

• Move backlog items into the sprint and start the sprint.

• Finally show the Screenshot of the sprint board at the start and end of the sprint.

Exp 10: Use the following requirements for a Library Management System:

• Add a feature to search books by title and author.

• Implement an online book reservation system.

• Generate monthly reports on borrowed books for administrators.

• Enable email notifications for overdue books.

• Add support for QR code scanning for borrowing and returning books.

• Create a user-friendly dashboard for librarians.

• Allow users to review and rate books.

• Integrate a chatbot for user assistance.

• Develop a mobile app version of the system.

• Provide multi-language support. Categorize each requirement using MOSCOW Method (Must-Have, Should-Have, Could-Have, or Won't-Have) based on the following criteria:

• Impact on the users and stakeholders.

• Feasibility considering time, budget, and resource constraints. Finally Submit the completed Google Sheet or Excel file with all requirements categorized and justified.

Exp 11: Link Jira tasks with Confluence to streamline task tracking and progress monitoring for the Library Management System development.

• Create a new page in Confluence titled "Library Management System Project Overview."

• Embed at least 5 Jira issues related to the development of the Library Management System (e.g., tasks from the sprint like "Develop book search functionality," "Create user login page," etc.).

• Use the Jira macro to display issues with status (e.g., "To Do," "In Progress," "Done").

• Add a progress bar in the Confluence page to visually track the completion of each embedded Jira task (e.g., percentage of tasks completed in the sprint).

• Submit a screenshot of the Confluence page showing the embedded Jira tasks and the progress bar.

Exp 12: You are designing a Task Management System for a small team. The system should include the following features:

1. User Login and Role Assignment

2. Task Creation and Assignment

3. Task Prioritization and Deadlines

4. Progress Tracking and Reporting Prioritize these requirements using the MoSCoW and Kano models in Jira.

Exp 13: You are tasked with developing an Online Learning Platform. The platform should include the following functionalities:

1. Course Enrollment and Registration

2. Video Lecture Streaming

3. Interactive Quizzes and Assignments

4. Progress Tracking Dashboard

5. Peer-to-Peer Discussion Forums

6. Certificate Generation Use Jira to categorize and prioritize these requirements using MoSCoW and Kano techniques.

karthika R A

Exp 14: Demonstrate how to work collaboratively in Git/GitHub on a project using the fork-and-pull request workflow. Tasks:

1. Fork an existing public GitHub repository (e.g., a sample JavaScript or Python project).

2. Clone the forked repository to your local machine using Git.

3. Create a new branch for the feature or change you want to work on.

4. Make modifications or add new features (e.g., add a function, fix a bug, or update the README).

5. Commit your changes and push the branch to GitHub.

6. Go to the GitHub repository and create a pull request to merge your feature branch into the main branch.

7. Review the pull request and provide feedback on the changes. Respond to feedback by making additional commits to the feature branch if necessary.

8. Finally submit the link to the pull request along with a summary of the changes you made and how you collaborated.

9. Once the pull request is approved, merge it into the main branch.

Exp 15: Demonstrate how to work with Git branches and resolve merge conflicts when collaborating with others. Tasks:

1. Clone a shared repository to your local machine.

2. Create a new branch and switch to it.

3. Make changes to a file (e.g., update a README or modify code in a specific function).

4. Commit your changes.

5. Push the changes to the remote repository.

6. Before merging, pull the latest changes from the main branch.

7. Switch back to your feature branch.

8. Merge the main branch into your feature branch.

9. If there are conflicts, resolve them manually by editing the conflicting files. After resolving conflicts, mark the conflicts as resolved. 10. Commit the resolved merge. 11. Push your feature branch with the merged changes to GitHub and create a pull request. 12. Submit a summary of the steps you performed, the conflicts you encountered, and how you resolved them.

Exp 16: Create a Static Website and Containerize, Build & Serve it using Docker. Tasks:

1. Create a Simple Static Website (index.html file) with basic HTML content.

2. Write/create a Dockerfile to serve the website using Nginx.

3. Build the Docker Image

4. Run the container:

5. Access the Website using a Browser

Exp 17 Create a Simple Python Flask API, Containerize the Application, Build & Push the Image using Docker and Deploy the Application using Kubernetes. Tasks:

1. Create a Simple Flask API by writing a Python file (app.py) with basic endpoints.

2. Containerize the Flask App using Dockerfile.

3. Build the Image using Docker.

4. Push the Image to Docker Hub.

5. Create Kubernetes manifests (Deployment YAML & Service YAML) to deploy the application. Apply the Manifests and Access the API via NodePort.

Exp 18: Set up a CI/CD pipeline to automate the building, testing, and deployment of a containerized application. Tasks:

1. Set up Jenkins on a local machine or server.

2. Create a Dockerfile to containerize a sample application.

3. Write a Jenkinsfile to automate the process of building the Docker container, running tests, and deploying to a cloud platform (e.g., AWS or GCP).

4. Configure Jenkins to trigger builds upon code commits or pull requests.

Exp 19: 14 Implement Continuous Deployment using GitHub Actions to deploy a Dockerized application. Tasks:

1. Set up a GitHub repository and push a simple Dockerized application.

2. Create a GitHub Actions workflow to automatically build and push the Docker image to Docker Hub or GitHub Container Registry.

3. Automate deployment to a cloud platform (e.g., AWS ECS, Azure Kubernetes Service, or Google Kubernetes Engine).

4. Test the CI/CD pipeline by pushing new code changes and verifying that the deployment occurs automatically.

Exp 20: 15 Create a GitHub Repository and Implement Version Control Tasks:

- Set up a repository for a team project.
- Create branches for individual modules.
- Merge branches with pull requests after code reviews.
- Resolve conflicts during branch merging.
- Document the workflow using the README file. • Submit the repository link

Exp 21

Containerize a Python Flask Application Using Docker

Tasks:

1. Write a simple Flask To-Do application.

2. Create Dockerfile.

3. Build and run Docker container.

4. Test application inside container.

5. Push Docker image to Docker Hub.

Exp 22

Push and Pull Docker Images Using Docker Hub

Tasks:

1. Create Docker image for static HTML website.

2. Push image to Docker Hub.

3. Pull image and run on another machine.

Exp 23

Deploy a Multi-Container Application Using Kubernetes

Tasks:

1. Create frontend, backend, and database containers.
2. Create Kubernetes YAML files.
3. Deploy application using kubectl.
4. Monitor pods and services.
5. Scale frontend container.

Exp 24

Create a CI/CD Pipeline Using GitHub Actions

Tasks:

1. Create GitHub repository for Flask application.
2. Create GitHub Actions workflow.
3. Build, test, and push Docker image.
4. Deploy application.

Exp 25

Create a GitHub Repository and Push Changes

Tasks:

1. Create GitHub repository.
2. Clone repository.
3. Edit README.md.
4. Stage, commit, and push changes.

Exp 26

Clone an existing GitHub repository to your local machine

and make a small change in one of the files.

Tasks:

1. Choose an existing public GitHub repository.

2. Clone the repository to local machine.

3. Open one file and make a small change.

4. Save the changes.

Exp 27

After modifying a file in a cloned repository, commit the changes locally and push them to GitHub.

Tasks:

1. Clone repository.

2. Modify README.md file.

3. Stage changes using git add.

4. Commit changes.

5. Push changes to GitHub.

Exp 28

Pull the latest changes from GitHub to keep the local repository updated.

Tasks:

1. Clone repository.

2. Collaborator makes change and pushes to GitHub.

3. Pull latest changes.

4. Verify updated files locally.

Exp 29

Create a new branch called feature-login, add a login function, and merge it into main branch.

Tasks:

1. Create feature-login branch.

2. Add login.py file.

3. Commit and push changes.

4. Create and merge pull request.

Exp 30

Fork a repository, create a feature branch, implement

changes, and submit a pull request.

Tasks:

1. Fork repository.

2. Clone forked repository.

3. Create feature branch.

4. Implement changes.

5. Push changes and submit pull request.